Abstract: Batch-Datenarchitektur für Taxinachfrage

Ziel war eine reproduzierbare Dateninfrastruktur für eine ML-Anwendung, die quartalsweise ein Modell zur Taxinachfrage in New York neu trainiert. Die Anwendung selbst gehört nicht zum Projekt. Quelle sind die monatlich per HTTPS veröffentlichten Yellow Taxi Trip Records der NYC TLC, rund 9,5 Millionen Fahrten pro Quartal. Die Umsetzung folgt der Medallion-Architektur: Rohdaten in Bronze, bereinigt und validiert in Silver, aggregiert zur stündlichen Nachfrage je Pickup-Zone in Gold. Jede Aufgabe läuft als eigener Container, die Infrastruktur steht als Compose-Datei im Repo, ein vollständiger Lauf startet mit dem Befehl „docker compose up“. Als vorausgesetzte Software ist lediglich Docker nötig, das über Containerisierung die Abhängigkeiten kapselt.

Umsetzung (Making-of)

In der Konzeptionsphase wurden Technologien recherchiert, die Datenquelle gewählt und ein Architekturentwurf mit Diagramm erstellt: TLC-Daten in einer Medallion-Architektur mit Microservices für Ingestion und Spark, gesteuert durch ein Orchestrierungsskript.

In der Erarbeitungsphase wurde dieser Entwurf implementiert. Hierfür wurde SeaweedFS als Data Lake genutzt (S3-API, ein Bucket pro Schicht). Die Python-Ingestion lädt die Monatsdateien des konfigurierten Quartals mit Retries und hält je Monat ein Manifest mit SHA-256 fest. Job 1 (Bronze →Silver) reduziert auf einem Spark-Cluster aus Master und zwei Workern die Spalten, vereinheitlicht Typen, entfernt exakte Duplikate und prüft zehn Qualitätsregeln. Verletzende Zeilen landen mit Ablehnungsgrund in der Quarantäne. Job 2 (Silver →Gold) bildet je Zone und Stunde Fahrtenzahl, Mittelwerte und Kalendermerkmale. Ein Laufbericht zählt Zeilen je Schicht und misst Latenzen.

Entwickelt wurde nach einem schlanken Git-Flow mit Feature-, Entwicklungs- und Hauptzweig. GitHub Actions prüft jeden Pull Request mit Lint und Unit-Tests, Merges und Releases zusätzlich mit einem End-to-End-Test, der zweimal auf demselben Lake die Idempotenz prüft. Eine Konzeptentscheidung wurde verworfen: Statt des Orchestrierungsskripts verkettet Docker Compose die Schritte selbst. Das vereinfacht die Architektur, verbessert die Reproduzierbarkeit und senkt die Voraussetzungen auf Docker. Abbildung 1 zeigt die finale Architektur.

Erfüllung der Anforderungen

Verlässlich ist das System durch Idempotenz: Alle Schichten sind nach Jahr und Monat partitioniert, jeder Job überschreibt genau die verarbeiteten Monate. Healthchecks und abgeschaltete automatische Neustarts stoppen die Kette bei Fehlern sichtbar, der Laufbericht macht die Vollständigkeit prüfbar. Skaliert wird über weitere Spark-Worker und die unabhängig verarbeiteten Monatspartitionen. Wartbar bleibt es durch einen Container und ein kurzes Skript je Aufgabe, gepinnte Images, gesperrte Abhängigkeiten, Unit- und End-to-End-Tests und Linting in CI.

Datensicherheit folgt dem Least-Privilege-Prinzip mit getrennten S3-Identitäten: Ingestion schreibt nur Bronze, Spark liest Bronze und schreibt Silver und Gold, die ML-Anwendung liest nur Gold, die Worker haben keine Zugangsdaten. Zugangsdaten liegen nur in einer nicht versionierten Umgebungsdatei, Dienste laufen bis auf den Laufbericht ohne Root, Ports sind nur lokal erreichbar. Datenschutz wird durch Datenminimierung in Silver erreicht, Governance durch Lineage über Manifest, Herkunftsdatei und Verarbeitungszeit je Zeile, Quarantäne und Laufbericht.

Ergebnis

Das Ziel ist erreicht: Ein einziger Compose-Aufruf lädt ein Quartal, bereinigt und aggregiert es und stellt Gold für die ML-Anwendung bereit, die es per pandas liest. Ein Beispiel-Notebook zeigt, dass sie Gold einlesen, verarbeiten und visualisieren kann, Silver aber nicht lesen darf.

Die Microservice-Architektur ist modular: Jeder Dienst erfüllt genau eine Aufgabe und lässt sich einzeln austauschen, testen und skalieren. Da das gesamte System als Infrastructure as Code in der Compose-Datei beschrieben ist, lässt es sich versioniert verwalten und auf jedem Rechner mit Docker identisch aufsetzen.

Reflexion

Probleme gab es vor allem beim Setup mit Podman, etwa bei der Portweiterleitung. Es waren bekannte GitHub-Issues mit dokumentierten Lösungen, die im Troubleshooting der README stehen.

Verbessern ließen sich Datensicherheit durch TLS, Verschlüsselung im Speicher und Secret-Management statt Umgebungsdatei, Governance und Verlässlichkeit durch Delta Lake oder Iceberg mit Versionen und atomaren Commits sowie einen Datenkatalog. Bei wachsenden Anforderungen käme ein Orchestrator wie Airflow für Planung und Monitoring infrage, dazu weitere Qualitätsregeln in den Tests. Im nächsten Projekt würde ich Tests und CI früher einplanen und die Container-Laufzeit von Anfang an festlegen.

Technisch nehme ich Spark und die Medallion-Architektur mit, dazu Containerisierung mit Docker Compose sowie die Python-Ingestion mit SeaweedFS und Datenqualitätsprüfungen. Als Soft Skills nehme ich das Zerlegen eines großen Ziels in aufbauende Schritte, die Visualisierung einer komplexen Architektur und die Bereitschaft mit, eigene Konzeptentscheidungen begründet zu verwerfen. Entscheidend für den Projekterfolg waren die gründliche Recherche in der Konzeptionsphase, reproduzierbare Abhängigkeiten mit uv, Unit- und End-to-End-Tests sowie strukturierte Git-Nutzung.

Ausblick: zweite Pipeline mit Stream-Verarbeitung

Da die TLC nur Monatsdateien veröffentlicht, müsste der Datenstrom selbst erzeugt werden. Ein Producer mit Leserecht auf Bronze würde die Fahrten zeitlich gestreckt und nach Abholzeit sortiert in ein Kafka-Topic schreiben, das Erzeugung und Verarbeitung entkoppelt.

Für die Integration kommen zwei Wege infrage. In einer Lambda-Architektur liefe der Stream als Speed Layer neben der Batch-Kette. Die Batch-Kette lieferte weiter die Trainingsdaten, der Stream machte die Nachfrage zwischen zwei Trainings sichtbar. In einer Kappa-Architektur ginge jede Verarbeitung über den Stream, Neuberechnungen spielten das Topic erneut ab. Das würde die bestehende Kette ersetzen und sich bei quartalsweisem Training kaum lohnen. Deshalb liegt Lambda hier näher.

Der Streaming-Job liefe als Compose-Dienst in eigenem Profil im vorhandenen Cluster, mit einer Kernobergrenze zugunsten der Batch-Jobs. Er würde die Regeln aus Job 1 nutzen, ausgelagert in ein gemeinsames Modul und vom Monatsbezug gelöst. Innerhalb eines Watermarks würde er Duplikate filtern und Fahrten je Zone in stündlichen Fenstern zählen. Ergebnisse und Checkpoints lägen in eigenen Lake-Pfaden mit eigener S3-Identität, Kafka-Zugriffsrechte schützten das Topic; Rechtekonzept und Lineage blieben erhalten. Nach einem Neustart liefe der Job verlustfrei weiter, solange das Topic die Ausfallzeit überbrückt.

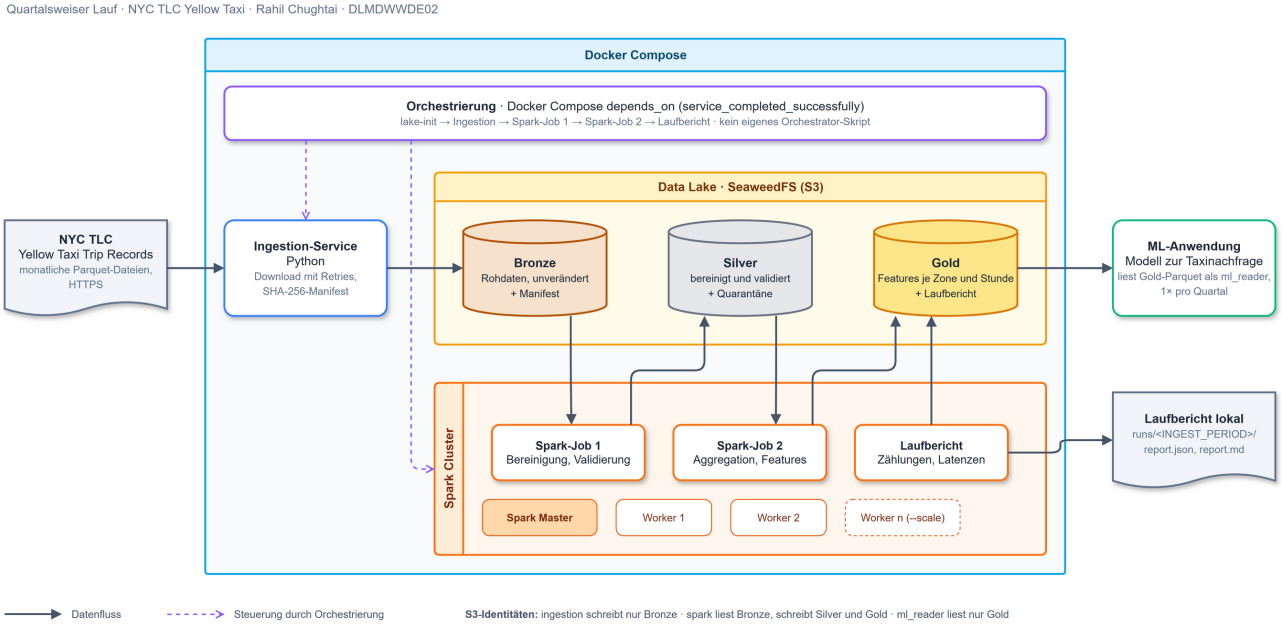


Abbildung 1: Finale Architektur